



DECEMBER 2, 2025

VULNERABILITY ASSESMENT

MODULE 05

VYANKATESH ANDIL



TABLE OF CONTENTS

Contents

Vulnerability Assessment	1
Vulnerabilities Scoring Systems and Databases	2
Common Vulnerability System (CVSS)	2
fig. CVSS Calculator 4.0	3
Common Vulnerabilities and Exposure (CVE)	3
Fig. CVE's Search Results	4
National Vulnerability Database (NVD)	4
Fig. NVD's Search Result	4
Relationship Between CVE and NVD	5
Common Weakness Enumeration	5
Fig. CWE's Search Result	6
1. Vulnerability Research with Vulnerability Scoring System and Databases	7
1.1 Objective	7
1.2 Scope	7
1.3 Methodology	7
1.4 Results	8
fig. 1.4.1 Searching for DNS Related Vulnerabilities	9
fig. 1.4.2 Detailed Information About the Vulnerability	9
fig. CWE Top 25 (2024) List	10
1.5 Results	10
1.6 Impact	11
1.7 Recommendations	11
1.8 Conclusion	11
2. VA Using Various VA Tools	12
2.1 Objective	12
2.2 Scope	12
2.3 Methodology	12
fig. 2.3.1 Performing Scanning using SmartScanner	13

TABLE OF CONTENTS

Fig . 2.3.2 Performing scanning using Nessus	14
fig. 2.3.3 Scanning vulnerabilities using Acuneticx	16
fig. 2.3.4 Performing Scanning using Microsoft Baseline security Analyzer	17
fig.2.3.5 Performing Scanning using Zaproxy	18
2.4 Tools Used	18
2.5 Results Summary	19
2.6 Impact Analysis	19
2.7 Recommendations	20
2.8 Conclusion	20
3. VA Using AI	21
3.1 Objective	21
3.2 Scope	21
3.3 Methodology	21
fig. 3.3.1 Using Gemini-cli for searching for the command	22
fig. 3.3.2 Getting Information about the misconfigurations	23
3.4 Findings / Results	23
3.5 Impact	23
3.6 Recommendations	24
3.7 Conclusion	24

VULNERABILITY ASSESSMENT

Vulnerability Assessment

Vulnerability Assessment is an examination of the ability of a system or application, Including current security procedures and controls, to withstand an assault. Vulnerability research is a process of discovering vulnerabilities and design flows that leave an OS and its applications open to attack or Misuse.

Vulnerability Classification

Vulnerability Type	Description	Examples
Misconfigurations/Weak Configurations	- Misconfigurations occur when systems, applications, or devices are not configured correctly, leaving them susceptible to exploitation - It allows attackers to break into a network and gain unauthorized access to systems	Network Misconfigurations Insecure protocols, open ports and services, errors, and weak encryption
		Host Misconfigurations Open permissions and unsecured root accounts
Application Flaws	- Application flaws are vulnerabilities in applications that are exploited by attackers - Flawed applications pose security threats such as data tampering and unauthorized access to configuration stores	Buffer overflows, memory leaks, resource exhaustion, integer overflows, null pointer/object dereference, DLL injection, race conditions, improper input handling, improper error handling, and code signing weakness
Poor Patch Management	- Software vendors provide patches that prevent exploitations and reduce the probability of threats exploiting a specific vulnerability - Unpatched software can make an application, server, or device vulnerable to various attacks	Unpatched servers, unpatched firmware, unpatched OS, and unpatched applications
Design Flaws	Logical flaws in the functionality of the system are exploited by the attackers to bypass the detection mechanism and acquire access to a secure system	Incorrect encryption and poor validation of data
Third-Party Risks	Third-party services can have access to privileged systems and applications, through which financial information, customer and employee data, and processes in the enterprise's supply chain can be compromised	Vendor management, supply-chain risks, outsourced code development, data storage, and cloud-based vs. on-premises risks

Vulnerabilities present in the system or network are classified into the following categories:

1. Misconfigurations/Weak Configurations:
 - a. Network Misconfigurations
 - i. Insecure Protocols
 - ii. Open ports and services
 - iii. Errors
 - iv. Weak Encryption
 - b. Host Misconfigurations
 - i. Open permissions
 - ii. Unsecured Accounts
2. Application Flaws
 - a. Buffer Overflows
 - b. Memory Leaks
 - c. Resource Exhaustion
 - d. Integer Overflows
 - e. Null Pointer/Object Defense
 - f. DLL Injection
 - g. Race Conditions
 - h. Improper Input Handling
 - i. Improper Error handling
 - j. Code Singing Weakness

VULNERABILITY ASSESSMENT

3. Poor Patch management
 - a. Unpatched Servers
 - b. Unpatched Firmware
 - c. Unpatched OS
 - d. Unpatched applications
4. Design Flows
5. Third-party risks
 - a. Vendor Management
 - i. System Integration
 - ii. Lack of Vendor Support
 - b. Supply Chain Risk
 - c. Outsourced Code Development
 - d. Data Storage
 - e. Cloud Based vs. on-premises risks
6. Default Installation/ Default Configurations
7. Operating System Flows
8. Default Passwords
9. Zero Day Vulnerabilities
10. System Sprawl/ Undocumented Assets
11. Improper Certificate and Key Management

VULNERABILITIES SCORING SYSTEMS AND DATABASES

- Common Vulnerability System (CVSS)
- Common Vulnerabilities and Exposures (CVE)
- National vulnerability Database (NVD)
- Common Weakness Enumeration (CWE)

Common Vulnerability System (CVSS)

Link: <https://www.first.org/cvss/calculator/4-0>

SERVERTY	BASE SCORE RANGE
None	0.0
Low	0.1-3.9
Medium	4.0-6.9
High	7.0-8.9
Critical	9.0-10.0

VULNERABILITY ASSESSMENT

fig. CVSS Calculator 4.0

Common Vulnerability Scoring System (CVSS-SIG)

- Calculator
- Specification Document
- User Guide
- Examples
- Frequently Asked Questions
- CVSS v4.0 Documentation & Resources
- CVSS v3.1 Archive
- CVSS v3.0 Archive
- CVSS v2 Archive
- CVSS v1 Archive
- JSON & XML Data Representations
- CVSS On-Line Training Course
- Identity & logo usage

CVSS

Common Vulnerability Scoring System Version 4.0 Calculator

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/V:N/VA:N/SC:N/SI:N/SA:N

CVSS v4.0 Score: 0 / None ⊕

Exploitability Metrics

Attack Vector (AV):

Attack Complexity (AC):

Attack Requirements (AT):

Privileges Required (PR):

User Interaction (UI):

Vulnerable System Impact Metrics

Confidentiality (VC):

Integrity (VI):

Availability (AV):

Common Vulnerabilities and Exposure (CVE)

Link: <https://www.cve.org/>

A **CVE (Common Vulnerabilities and Exposures)** is a **publicly disclosed cybersecurity vulnerability** that has been assigned a unique identification number so security professionals worldwide can track, discuss, and fix it consistently.

- What a CVE Includes
 - CVE ID
 - Description of the vulnerability
 - Affected products
 - Severity score (CVSS)
 - References (advisories, reports, PoCs)
- What CVEs Are Used For
 - Vulnerability assessment
 - Penetration testing
 - Patch management
 - Risk analysis
 - Creating exploit modules in Metasploit
 - Compliance audits

VULNERABILITY ASSESSMENT

Fig. CVE's Search Results

The screenshot shows the CVE Search Results page. At the top, there is a navigation bar with links: About, Partner Information, Program Organization, Downloads, Resources & Support, and Report/Request. Below the navigation bar is a search bar with the text "CVE-2025" and a "Search" button. To the right of the search bar is a "Site Search" button. Below the search bar is a notice: "Notice: Expanded keyword searching of CVE Records (with limitations) is now available in the search box above. Learn more here." Below the notice is the "Search Results" section. It shows "Showing 1 - 25 of 84,944 results for CVE-2025". There are two dropdown menus: "Show: 25" and "Sort by: CVE ID (new to old)". Below the dropdowns are three search results:

- CVE-2025-9999** CNA: ARC Informatique
Some payload elements of the messages sent between two stations in a networking architecture are not properly checked on the receiving station allowing an attacker to execute unauthorized commands in...
[Show more](#)
- CVE-2025-9998** CNA: ARC Informatique
The sequence of packets received by a Networking server are not correctly checked. An attacker could exploit this vulnerability to send specially crafted messages to force the application to stop.
- CVE-2025-9997** CNA: Schneider Electric SE
CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') vulnerability exists that could cause command injection in BLMon that is executed in the

National Vulnerability Database (NVD)

Link: <https://nvd.nist.gov/>

The **NVD (National Vulnerability Database)** is the **U.S. government's official database of cybersecurity vulnerabilities**, built on top of the CVE system.

- Purpose of NVD
 - Help organizations understand how **severe** a vulnerability is
 - Provide structured data for automated vulnerability scanners
 - Assist in risk assessment and patch prioritization

Fig. NVD's Search Result

The screenshot shows the NVD Search Result page for CVE-2025-8714. At the top, there is a navigation bar with the NIST logo and a "NVD MENU" button. Below the navigation bar is the "NATIONAL VULNERABILITY DATABASE" header. Below the header is a "VULNERABILITY" button. Below the button is the "CVE-2025-8714 Detail" section. It shows "AWAITING ANALYSIS" and "This CVE record has been marked for NVD enrichment efforts." Below this is the "Description" section. It contains the following text: "Untrusted data inclusion in pg_dump in PostgreSQL allows a malicious supervisor of the origin server to inject arbitrary code for restore-time execution as the client operating system account running pgsql to restore the dump, via psql meta commands, pg_dumpall is also affected. pg_restore is affected when used to generate a plain-format dump. This is similar to MySQL CVE-2024-21096. Versions before PostgreSQL 17.6, 16.10, 15.14, 14.39, and 13.22 are affected." Below the description is the "Metrics" section. It shows "CVSS Version 4.0", "CVSS Version 3.x", and "CVSS Version 2.0". Below the metrics is the "NIST NVD" section. It shows "Base Score: 9.8" and "NVD assessment not yet provided." Below the NIST NVD section is the "CNA: PostgreSQL" section. It shows "Base Score: 9.8" and "Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/E:POA:M".

VULNERABILITY ASSESSMENT

Relationship Between CVE and NVD

CVE	NVD
Created by MITRE	Managed by NIST
Unique ID + brief description	Full vulnerability analysis
No severity score	CVSS score provided
No impact details	Detailed impact + exploit metrics
High-level only	Deep technical data

Common Weakness Enumeration

Link: <https://cwe.mitre.org/find/index.html>

Examples of CWEs:

CWE ID	Weakness Name	Explanation
CWE-79	Cross-Site Scripting (XSS)	User input is not sanitized
CWE-89	SQL Injection	Unvalidated queries sent to DB
CWE-200	Information Exposure	Sensitive data leaked
CWE-119	Buffer Overflow	Writing outside memory bounds
CWE-306	Missing Authentication	No auth mechanism present
CWE-522	Weak Password Storage	Plaintext passwords
CWE-601	Open Redirect	Unsafe URL redirection

VULNERABILITY ASSESSMENT

Fig. CWE's Search Result

The screenshot shows the CWE website interface. At the top, the logo for "Common Weakness Enumeration" is displayed, along with a tagline: "A community-developed list of SW & HW weaknesses that can become vulnerabilities". To the right, there are two circular badges: one for "Top 25" and another for "Top HW CWE". A link "New to CWE? Start here!" is also present. Below the header, a navigation bar includes links for "Home", "About", "Learn", "Access Content", "Community", and "Search". A search bar is located on the right side of the navigation bar, with the text "ID Lookup:" and a "Go" button.

The main content area is titled "Search the CWE Web Site". Below this, a message states: "To search the CWE Web site, enter a keyword by typing in a specific term or multiple keywords separated by a space, and click the Google Search button or press return." A search input field contains the text "SMB", and a search button is visible to its right. Below the search bar, the results are displayed as follows:

About 26 results (0.10 seconds)

- CWE-284: Improper Access Control (4.18)**
cwe.mitre.org > CWE List
Vulnerability Mapping: DISCOURAGED This CWE ID should not be used to map to real-world vulnerabilities. Abstraction: Pillar Pillar - a weakness that is the ...
- CWE-693: Protection Mechanism Failure (4.18)**
cwe.mitre.org > CWE List
Vulnerability Mapping: DISCOURAGED This CWE ID should not be used to map to real-world vulnerabilities. Abstraction: Pillar Pillar - a weakness that is the ...
- CWE-319: Cleartext Transmission of Sensitive Information**
cwe.mitre.org > CWE List
This allows cloud storage resources to successfully connect and transfer data without the use of encryption (e.g., HTTP, SMB 2.1, SMB 3.0, etc.). Azure's ...
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor**
cwe.mitre.org > CWE List
Vulnerability Mapping: DISCOURAGED This CWE ID should not be used to map to real-world vulnerabilities. Abstraction: Class Class - a weakness that is ...
- CWE-427: Uncontrolled Search Path Element (4.18)**
cwe.mitre.org > CWE List
the current working directory. In some cases, the attack can be conducted remotely, such as when SMB or WebDAV network shares are used. One or more locations in ...

- **CVE** = actual vulnerability found in a specific product
- **CWE** = the type/category of weakness that caused vulnerability

1.VULNERABILITY RESEARCH WITH VULNERABILITY SCORING SYSTEM AND DATABASES

1.Vulnerability Research with Vulnerability Scoring System and Databases

Vulnerability research is the process of identifying, analyzing, and understanding weaknesses within systems, applications, or networks. The goal is to determine how vulnerability works, its potential impact, and the methods used for detection and mitigation. To standardize this process, global vulnerability scoring systems and databases—such as CVE, CVSS, NVD, and CWE—provide structured ways to classify and assess security issues.

1.1 OBJECTIVE

The primary objective of this activity is to:

- Understand how vulnerability research is performed using standard security databases.
- Learn how vulnerabilities are cataloged using identifiers such as CVE and CWE.
- Analyze vulnerabilities using the **Common Weakness Enumeration (CWE)** classification.
- Use vulnerability scoring systems like CVSS to understand severity and impact.
- Gain practical insight into how organizations track and remediate known weaknesses.

1.2 SCOPE

- Understanding CWE categories and weak descriptions.
- Navigating vulnerability databases such as MITRE, NVD, and CVE lists.
- Mapping vulnerabilities to CWE IDs.
- Analyzing severity using **CVSS v3.x** scoring metrics.
- Reviewing publicly disclosed vulnerabilities and extracting technical details.
- Using CWE to understand root causes behind vulnerabilities

1.3 METHODOLOGY

Identifying Vulnerabilities

Used public vulnerability databases such as

- **CVE (Common Vulnerabilities and Exposures)**
- **NVD (National Vulnerability Database)**
- **MITRE CWE Database**

Mapping Vulnerabilities to CWE

- Reviewed CVE entries on NVD or MITRE.
- Examined the root cause section to identify related CWE ID(s).
- Checked CWE descriptions to understand

1.VULNERABILITY RESEARCH WITH VULNERABILITY SCORING SYSTEM AND DATABASES

- Weakness type
- Attack vector
- Affected components
- Common mistakes by developers

Severity Analysis using CVSS

For each vulnerability, CVSS v3.1 scoring parameters were analyzed:

Base Score: Attack vector, complexity, privileges required, user interaction

Impact Metrics: Confidentiality, integrity, availability

Researching Affected Products and Impact

- Identified vendors and affected versions.
- Reviewed exploitability level (Low / Medium / High).
- Verified references such as security advisories, PoCs, and exploit write-ups

Documentation:

- CVE ID
- CWE ID(s)
- Description and root cause
- Severity score (CVSS)
- Mitigation and patch availability
- References from vendor advisories

1.4 RESULTS

The vulnerability research process followed a structured methodology

Identifying Vulnerabilities

- Used public vulnerability databases such as
 - CVE (Common Vulnerabilities and Exposures)
 - NVD (National Vulnerability Database)
 - MITRE CWE Database
- Selected vulnerabilities based on technology (e.g., Windows, Linux, Web Applications, Protocols).

Mapping Vulnerabilities to CWE

- Reviewed CVE entries on NVD or MITRE
- Examined the root cause section to identify related CWE ID(s)
- Checked CWE descriptions to understand
 - Weakness type

1.VULNERABILITY RESEARCH WITH VULNERABILITY SCORING SYSTEM AND DATABASES

- Attack vector
- Affected components
- Common mistakes by developers

We are searching for the vulnerabilities of the running services that were found in the target system in the previous module Enumeration.

The screenshot shows the CWE Common Weakness Enumeration website. The search bar contains the text 'dns'. Below the search bar, there are search results for 'CWE-292: DEPRECATED: Trusting Self-reported DNS Name', 'CWE-247: DEPRECATED: Reliance on DNS Lookups in a Security Decision', 'CWE-350: Reliance on Reverse DNS Resolution for a Security Decision', and 'CWE-435: Improper Interaction Between Multiple Correctly-Behaving ...'. The results are listed with their respective descriptions and links to view more information.

fig. 1.4.1 Searching for DNS Related Vulnerabilities

The screenshot shows the detailed information for CWE-247: DEPRECATED: Reliance on DNS Lookups in a Security Decision. The page includes a description, vulnerability mapping notes, usage, reason, rationale, and comments. It also features a content history table with columns for Submissions, Submission Date, Submitter, and Organization.

Submissions	Submission Date	Submitter	Organization
2006-07-19 (CWE Draft 3, 2006-07-19)	2006-07-19	7 Pernicious Kingdoms	

fig. 1.4.2 Detailed Information About the Vulnerability

Now Navigate to CWE List tab and go to CWE Top 25 (2024):

Access Content → CWE List → CWE Top 25 (2024)

1.VULNERABILITY RESEARCH WITH VULNERABILITY SCORING SYSTEM AND DATABASES

CWE VIEW: Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses

View ID: 1430
Vulnerability Mapping: PROHIBITED
Type: Graph

Downloads: Booklet | CSV | XML

Objective
CWE entries in this view are listed in the 2024 CWE Top 25 Most Dangerous Software Weaknesses.

Audience

Stakeholder	Description
Software Developers	By following the CWE Top 25, developers are able to significantly reduce the number of weaknesses that occur in their software.
Product Customers	Customers can use the weaknesses in this view in order to formulate independent evidence of a claim by a product vendor to have eliminated / mitigated the most dangerous weaknesses.
Educators	Educators can use this view to focus curriculum and teachings on the most dangerous weaknesses.

Relationships
The following graph shows the tree-like relationships between weaknesses that exist at different levels of abstraction. At the highest level, categories and pillars exist to group weaknesses. Categories (which are not technically weaknesses) are special CWE entries used to group weaknesses that share a common characteristic. Pillars are weaknesses that are described in the most abstract fashion. Below these top-level entries are weaknesses are varying levels of abstraction. Classes are still very abstract, typically independent of any specific language or technology. Base level weaknesses are used to present a more specific type of weakness. A variant is a weakness that is described at a very low level of detail, typically limited to a specific language or technology. A chain is a set of weaknesses that must be reachable consecutively in order to produce an exploitable vulnerability. While a composite is a set of weaknesses that must all be present simultaneously in order to produce an exploitable vulnerability.

Expand All | Collapse All

1430 - Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses

- Improper Neutralization of Input During Web Page Generation ("Cross-site Scripting") - (79)
- Out-of-bounds Write - (787)
- Improper Neutralization of Special Elements used in an SQL Command ("SQL Injection") - (89)
- Cross-Site Request Forgery (CSRF) - (352)
- Improper Limitation of a Pathname to a Restricted Directory ("Path Traversal") - (22)
- Out-of-bounds Read - (125)
- Improper Neutralization of Special Elements used in an OS Command ("OS Command Injection") - (78)
- Use After Free - (416)
- Missing Authorization - (862)
- Unrestricted Upload of File with Dangerous Type - (434)
- Improper Control of Generation of Code ("Code Injection") - (94)
- Improper Input Validation - (20)
- Improper Neutralization of Special Elements used in a Command ("Command Injection") - (77)
- Improper Authentication - (287)
- Improper Privilege Management - (268)
- Deserialization of Untrusted Data - (502)
- Exposure of Sensitive Information to an Unauthorized Actor - (200)
- Incorrect Authorization - (863)
- Server-Side Request Forgery (SSRF) - (918)
- Improper Restriction of Operations within the Bounds of a Memory Buffer - (119)
- NULL Pointer Dereference - (476)
- Use of Hard-coded Credentials - (798)
- Integer Overflow or Wraparound - (190)
- Uncontrolled Resource Consumption - (400)

fig. CWE Top 25 (2024) List

Severity Analysis using CVSS

For each vulnerability, CVSS v3.1 scoring parameters were analyzed

- **Base Score:** Attack vector, complexity, privileges required, user interaction
- **Impact Metrics:** Confidentiality, integrity, availability
- **Temporal/Environmental Scores:** Adjusted based on exploit maturity and environment

Researching Affected Products and Impact

- Identified vendors and affected versions.
- Reviewed exploitability level (Low / Medium / High).
- Verified references such as security advisories, PoCs, and exploit write-ups

Documentation

- CVE ID
- CWE ID(s)
- Description and root cause
- Severity score (CVSS)
- Mitigation and patch availability
- References from vendor advisories

1.5 RESULTS

The vulnerability research yielded insights into the structure and classification of software weaknesses:

1.VULNERABILITY RESEARCH WITH VULNERABILITY SCORING SYSTEM AND DATABASES

- Vulnerabilities are consistently categorized using **CWE**, making it easier to understand the underlying weakness rather than the specific instance.
- CVSS scoring enables prioritization based on exploitability and impact.
- CWE categories such as:
 - CWE-79 (Cross-Site Scripting)
 - CWE-89 (SQL Injection)
 - CWE-287 (Improper Authentication)
 - CWE-119 (Buffer Overflow)
 - CWE-200 (Information Disclosure)
- Most vulnerabilities could be traced back to development flaws such as improper validation, poor access control, or unsafe memory handling

1.6 IMPACT

- CWE classification allows security teams to understand systemic security weaknesses in software.
- Organizations can improve secure coding practices by identifying recurring CWE patterns.
- CVSS scores help prioritize remediation—critical vulnerabilities with high CVSS scores require urgent attention.
- Security research helps detect emerging threats and reduces risk exposure

1.7 RECOMMENDATIONS

- Implement secure coding practices aligned with CWE Top 25 and OWASP guidelines.
- Use static and dynamic analysis tools that detect CWE-related weaknesses.
- Train developers on common weakness patterns and secure development lifecycle (SDLC).
- Continuously review CVE/NVD databases for newly disclosed vulnerabilities.
- Patch systems promptly according to severity and exploit availability

1.8 CONCLUSION

Vulnerability research using CWE, CVE, and CVSS provides a standardized approach to understanding and assessing software weaknesses. CWE helps identify the root causes behind vulnerabilities, while CVSS gives a quantifiable severity score for prioritization. Using these structured databases and scoring systems enhances the overall security posture of an organization by promoting proactive detection and remediation of weaknesses.

2. VA USING VARIOUS VA TOOLS

2. VA Using Various VA Tools

Ethical hackers and Pentesters are aided in vulnerabilities assessments with the help of various tools that make vulnerability assessment an easy task.

Vulnerability Assessment (VA) is a systematic process of identifying, analyzing, and prioritizing security weaknesses in a system, application, or network. The goal is to help organizations proactively detect vulnerabilities before they can be exploited by attackers. In this assessment, multiple industry-recognized tools: Smart Scanner, Nessus, Acunetix, and MBSA: were used to provide a comprehensive security evaluation.

2.1 OBJECTIVE

- To identify security vulnerabilities in the target system, network, and web applications.
- To compare and validate findings using various vulnerability assessment tools.
- To analyze risk levels based on industry scoring systems such as **CVSS (Common Vulnerability Scoring System)**.
- To provide remediation recommendations to strengthen the security posture

2.2 SCOPE

The assessment covered the following areas:

- **Network vulnerabilities** using Smart Scanner and Nessus.
- **Web application vulnerabilities** using Acunetix.
- **Operating system and configuration vulnerabilities** using Microsoft Baseline Security Analyzer (MBSA).
- Scanning included open ports, OS misconfigurations, weak credentials, missing patches, insecure web components, and known CVEs.

2.3 METHODOLOGY

The vulnerability assessment followed standard security practices based on **OWASP**, **SANS**, and **NIST 800-115**

Information Gathering

- Identified active hosts and services.
- Collected system metadata and application details

Vulnerability Scanning

Each tool was used as follows:

2. VA USING VARIOUS VA TOOLS

1. Smart Scanner

- Performed automated reconnaissance
- Detected open ports, service versions, and basic vulnerability signatures.

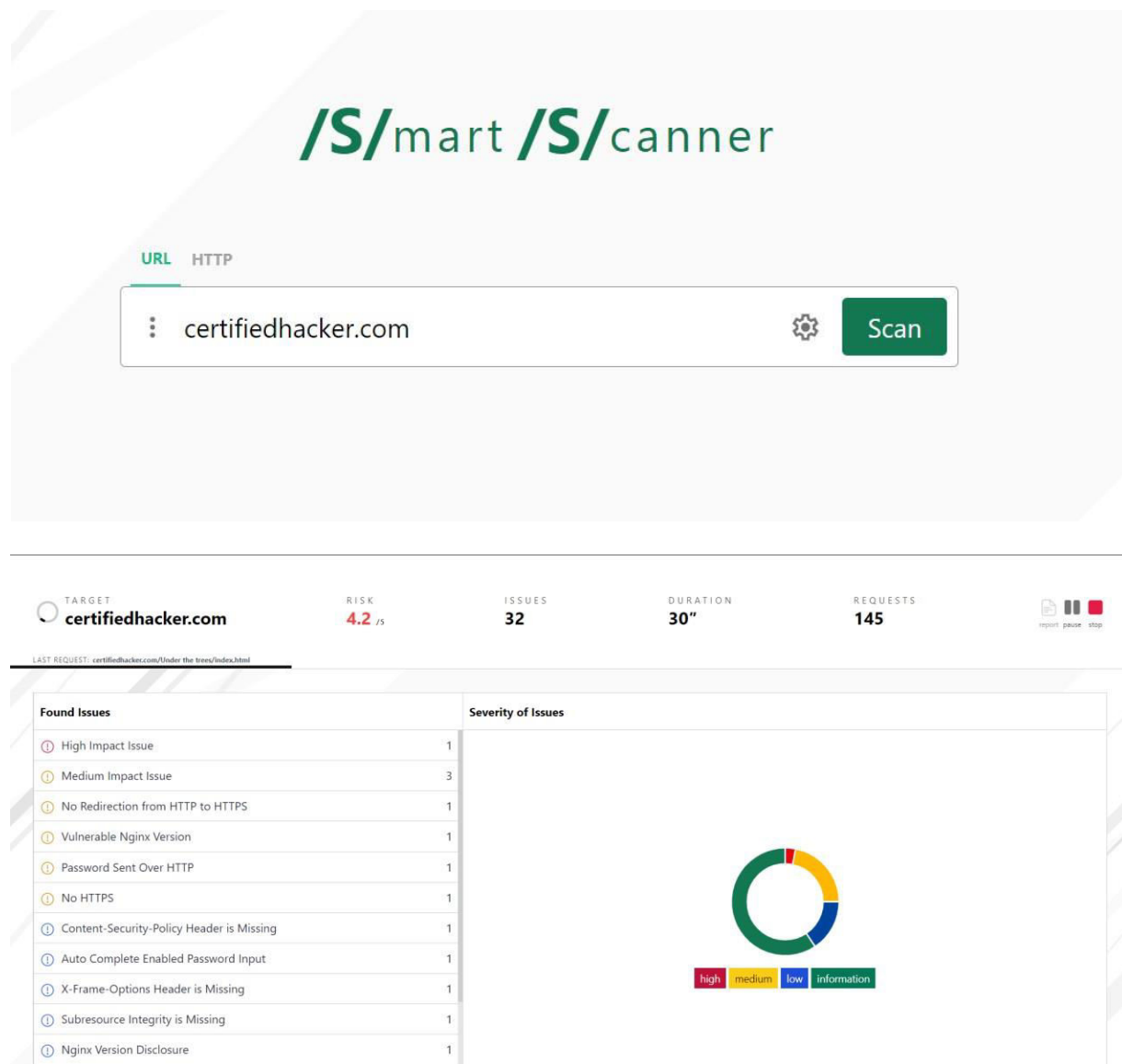


fig. 2.3.1 Performing Scanning using SmartScanner

- After this step save the report and Analyze Critical Vulnerabilities Manually

2. Nessus

- Conducted in-depth network scanning

2. VA USING VARIOUS VA TOOLS

- Identified vulnerabilities linked to CVEs, CWE weaknesses, and configuration issues
- Assigned **CVSS scores** for risk prioritization

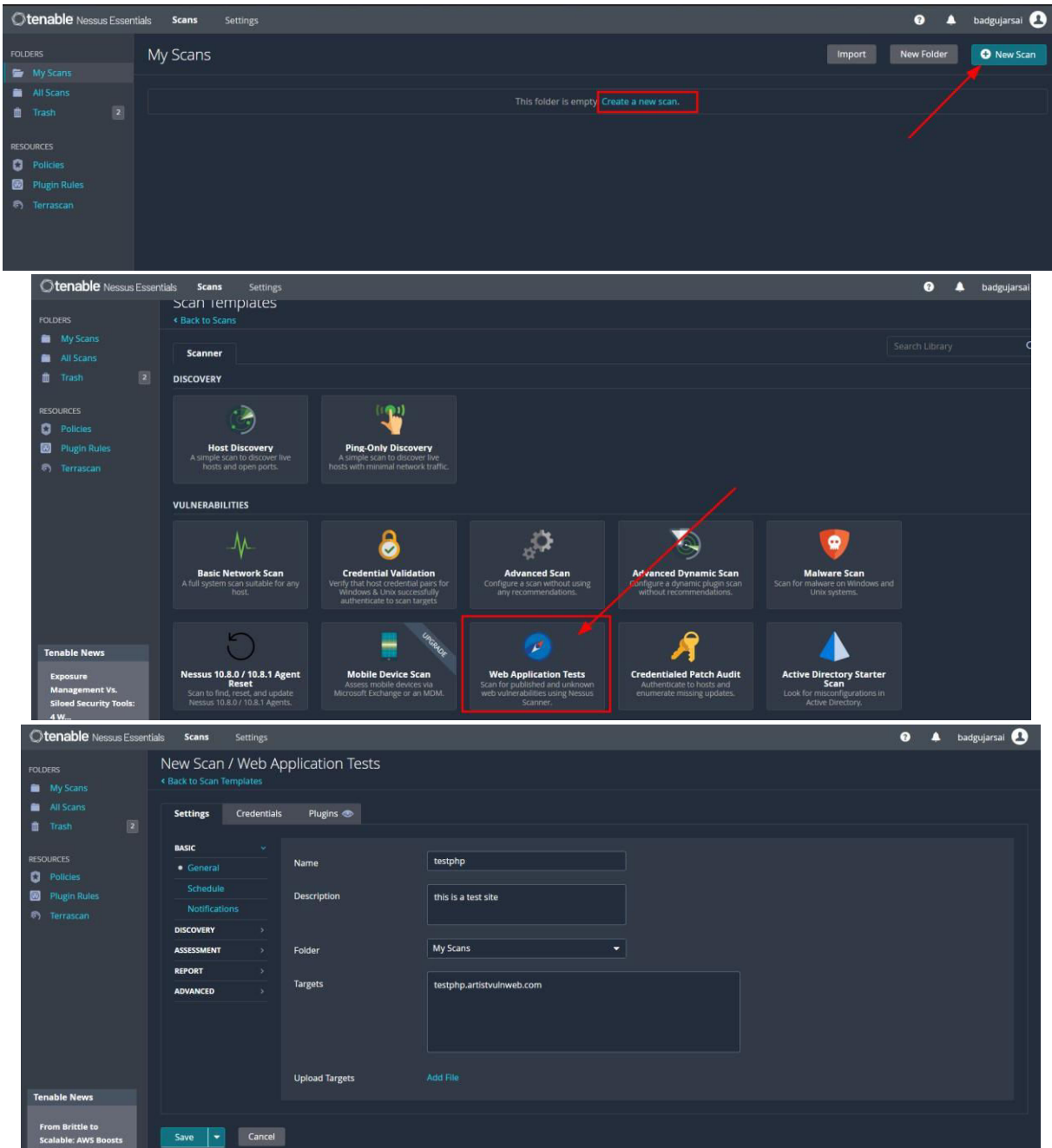


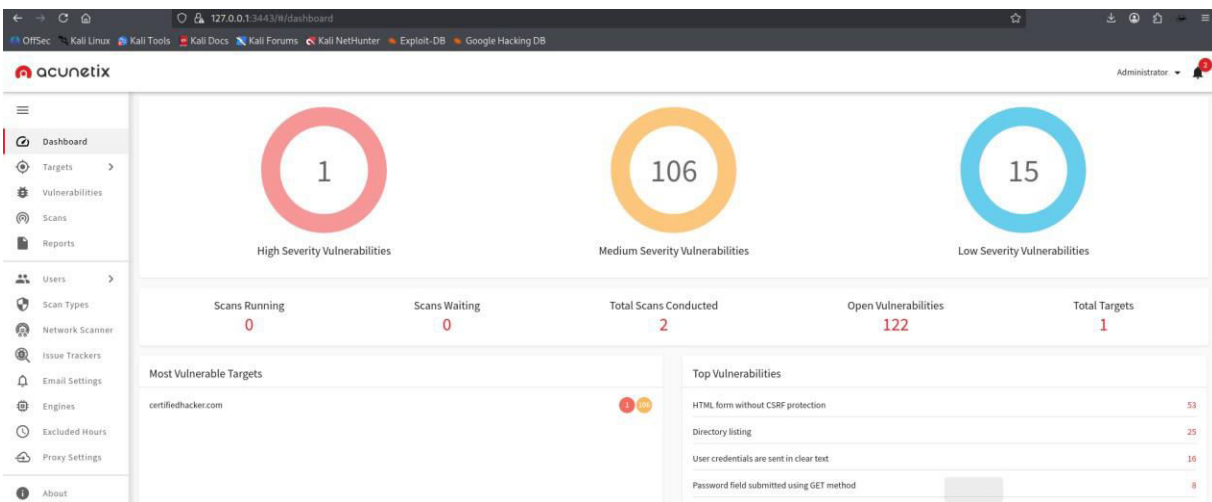
Fig . 2.3.2 Performing scanning using Nessus

Then for Start new scan click on save button and wait for the Scan to complete after that Download the report and then analyze the report and test critical vulnerabilities manually.

2. VA USING VARIOUS VA TOOLS

3. Acunetix

- Focused on dynamic web application scanning
- Detected injection flaws, authentication issues, misconfigurations, outdated libraries, XSS, CSRF, etc.
- Provided detailed proof-of-exploitation for critical findings.



acunetix

127.0.0.1:3443/#/vulnerabilities/status=open

Administrator

Vulnerabilities

BRF Export

Generate Report

Mark as

Refresh

Send To Issue Tracker

Status: Open

Severity	Vulnerability	URL	Parameter	Status	Confidence %	Last Seen
High	Possible database backup	http://certifiedhacker.com/		Open	95	Dec 3, 2025, 12:43:40 PM
Medium	Directory listing	http://certifiedhacker.com/js/		Open	100	Dec 3, 2025, 12:43:01 PM
Medium	Directory listing	http://certifiedhacker.com/js/source/		Open	100	Dec 3, 2025, 12:43:05 PM
Medium	Directory listing	http://certifiedhacker.com/Online%20Booking/js/		Open	100	Dec 3, 2025, 12:45:43 PM
Medium	Directory listing	http://certifiedhacker.com/Online%20Booking/js/hotels/		Open	100	Dec 3, 2025, 12:45:46 PM
Medium	Directory listing	http://certifiedhacker.com/css/		Open	100	Dec 3, 2025, 12:47:57 PM
Medium	Directory listing	http://certifiedhacker.com/blog/		Open	100	Dec 3, 2025, 12:48:43 PM
Medium	Directory listing	http://certifiedhacker.com/docs/		Open	100	Dec 3, 2025, 12:50:10 PM
Medium	Directory listing	http://certifiedhacker.com/images/		Open	100	Dec 3, 2025, 12:51:12 PM

2. VA USING VARIOUS VA TOOLS

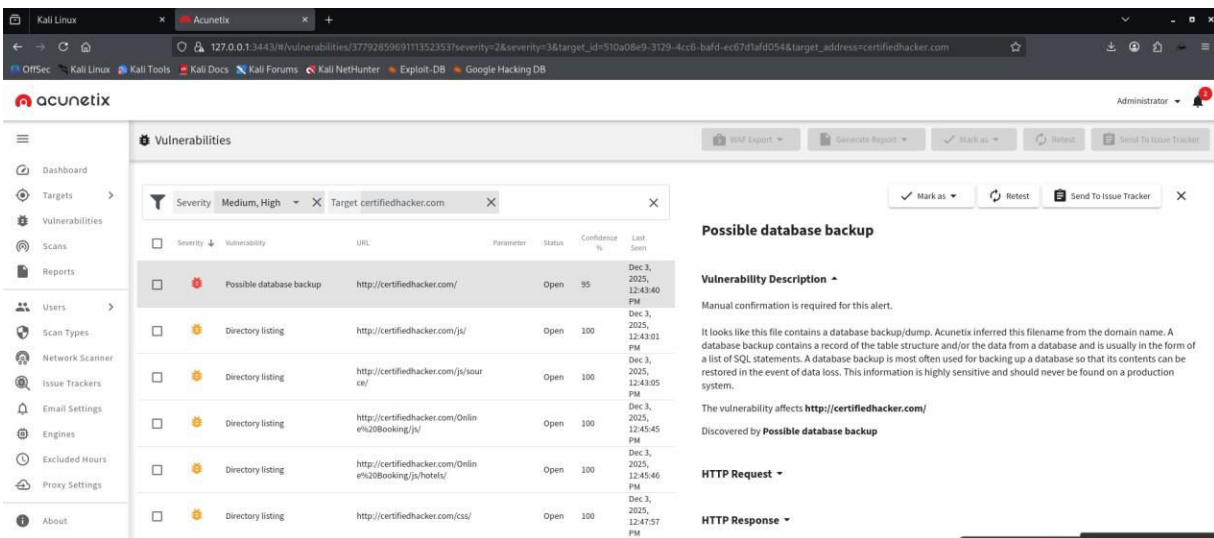
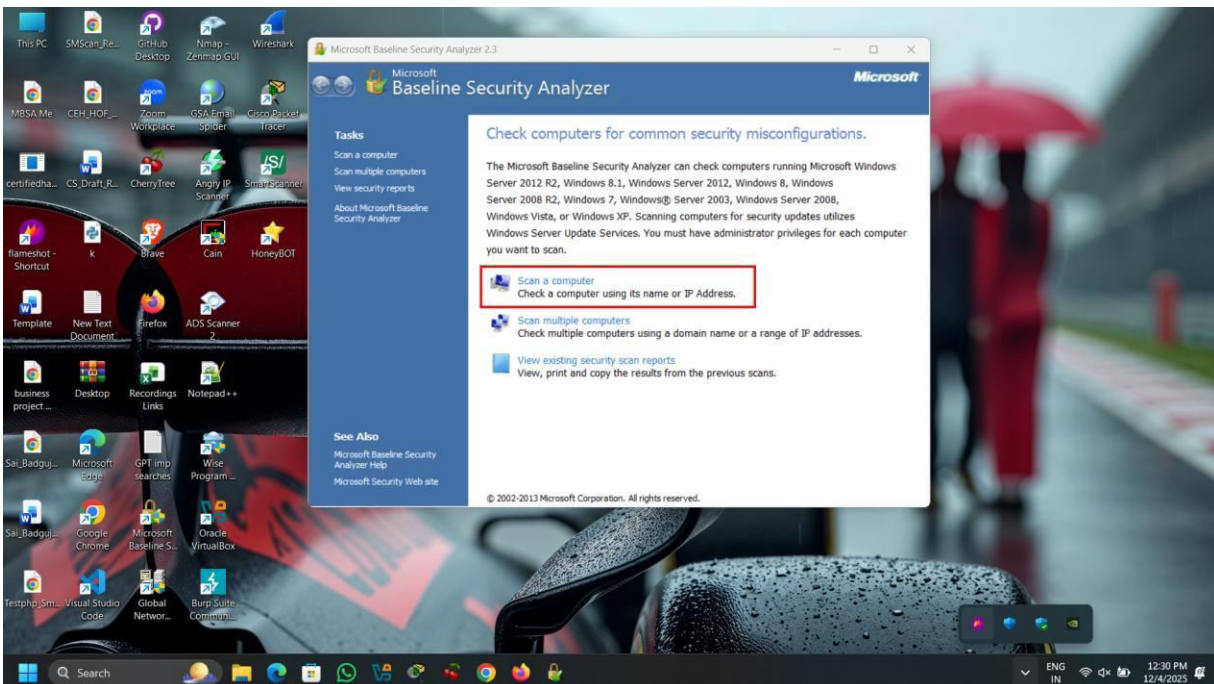


fig. 2.3.3 Scanning vulnerabilities using Acunetix

4. Microsoft Baseline Security Analyzer (MBSA)

- Assessed Windows-specific patching and configuration compliance.
- Identified weak account policies, missing security updates, and insecure system settings



Now then select the scan type and set target and start the scan

2. VA USING VARIOUS VA TOOLS

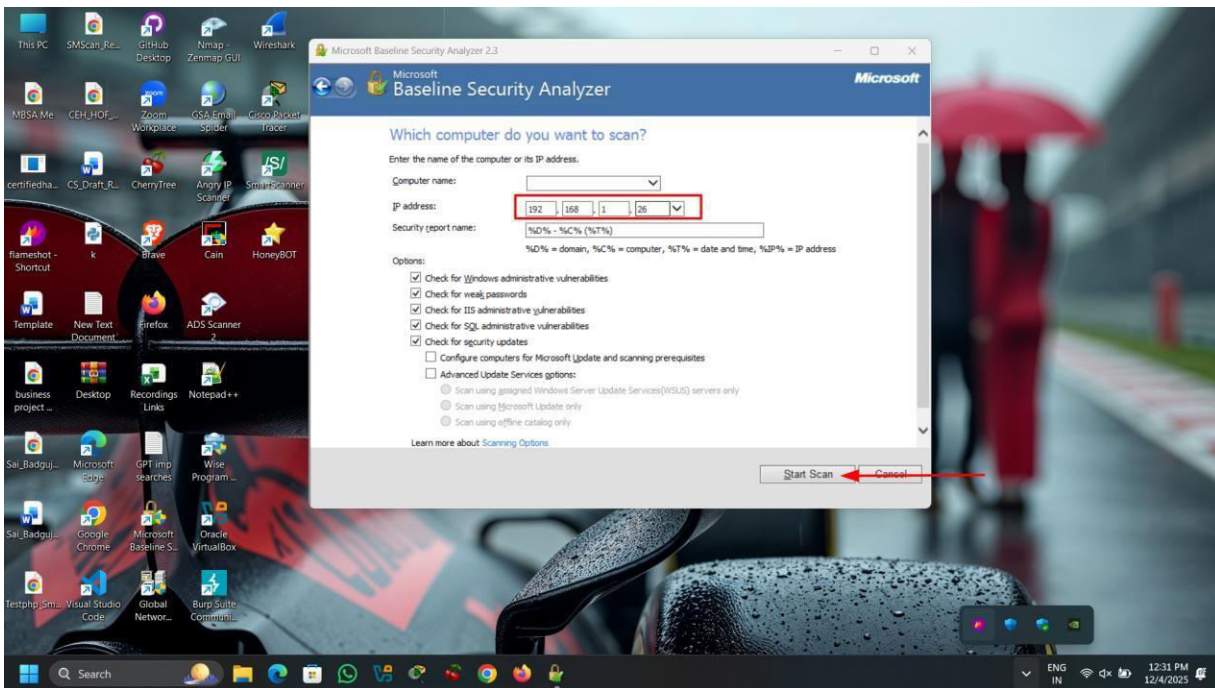
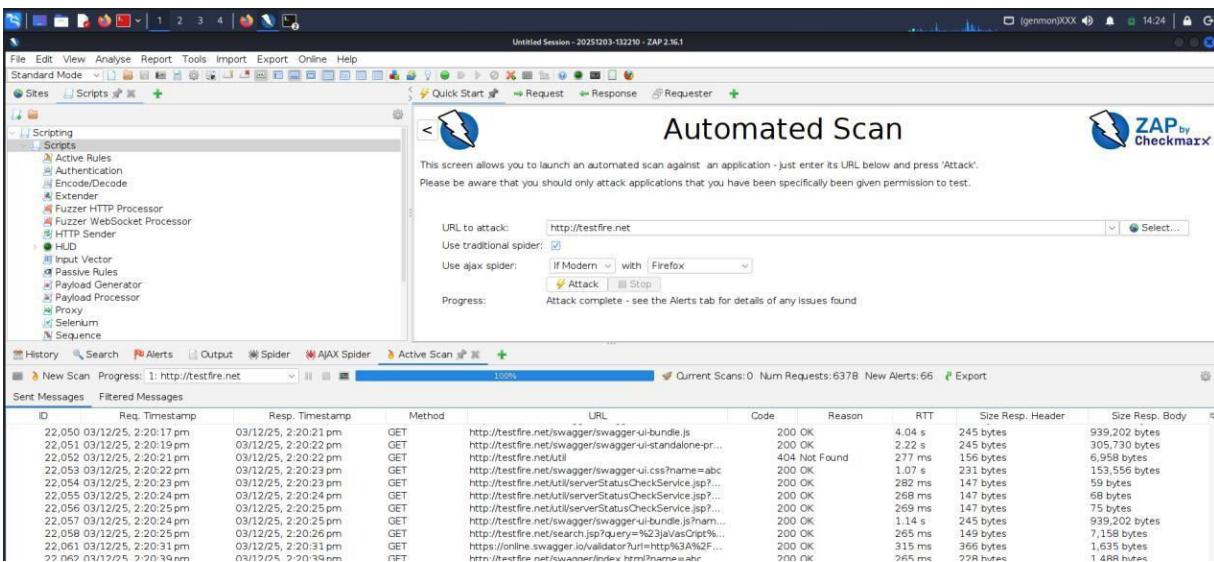


fig. 2.3.4 Performing Scanning using Microsoft Baseline security Analyzer

5. OWASP ZAP (ZAP Proxy)

- Used as a dynamic web application scanner to detect client-side and server-side vulnerabilities.



2. VA USING VARIOUS VA TOOLS

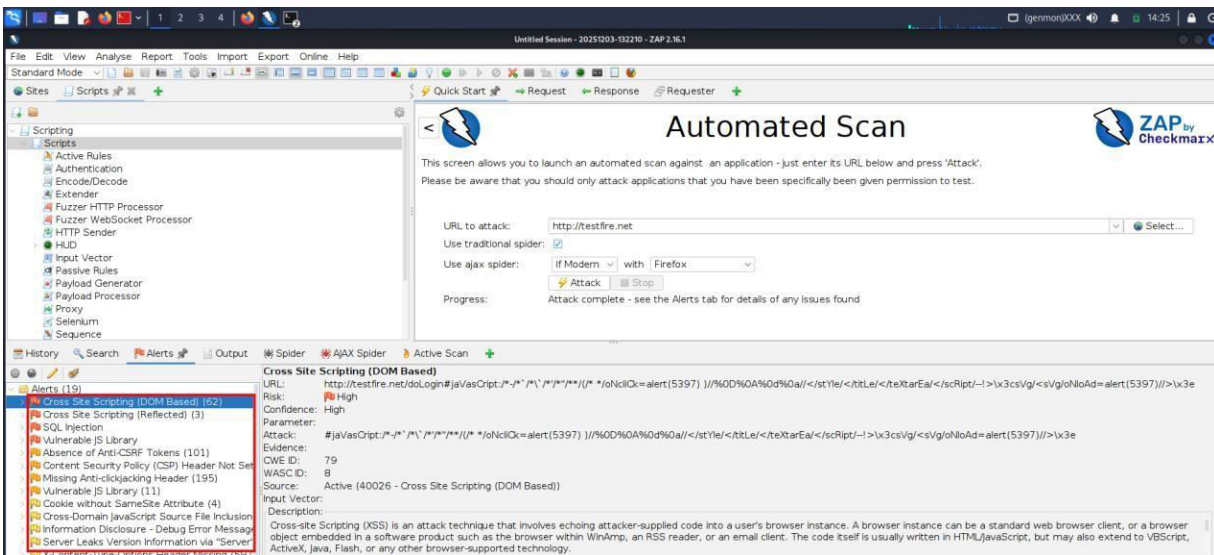


fig.2.3.5 Performing Scanning using Zaproxy

Verification & Analysis

- Verified critical findings manually where necessary.
- Correlated results across tools to reduce false positives.
- Classified vulnerabilities into severity: **Critical, High, Medium, Low** based on CVSS

Reporting

- Documented all findings with description, impact, affected components, and remediation steps.
- Provided risk rating and CVE/CWE mapping for each vulnerability

2.4 TOOS USED

1. Smart Scanner

- Lightweight vulnerability detection.
- Provides quick insights into open ports, services, and basic weaknesses

2. Nessus Essentials / Professional

- One of the world's most widely used VA tools.
- Detects network-based vulnerabilities, configuration issues, and compliance gaps.
- Uses the Nessus plugin database with mapped **CVE, CWE, CPE, & CVSS scoring**

3. Acunetix Web Vulnerability Scanner

- Specialized for web application vulnerabilities.

2. VA USING VARIOUS VA TOOLS

- Detects SQL Injection, XSS, Authentication Weaknesses, Server Misconfigurations, etc.
- Offers proof-of-concept payloads

4. Microsoft Baseline Security Analyzer (MBSA)

- Windows OS vulnerability and configuration scanner.
- Identifies missing patches, weak passwords, insecure settings, and compliance issues

5. OWASP Zap (ZAP Proxy)

- Proxy-based web vulnerability scanning

Tool	Purpose
Smart Scanner	Fast detection of open ports and network vulnerabilities
Nessus	Comprehensive vulnerability scanning and compliance checks
Acunetix	Automated web application security testing
Microsoft Baseline Security Analyzer (MBSA)	Windows patch and configuration validation
OWASP ZAP	Proxy-based web vulnerability scanning

2.5 RESULTS SUMMARY

- Smart Scanner identified **open ports**, exposed services, and version-related vulnerabilities.
- Nessus detected **critical CVE-linked vulnerabilities**, insecure configurations, and missing patches.
- Acunetix identified **web application flaws** such as outdated libraries, XSS potential, and weak authentication.
- MBSA reported **missing security updates**, weak account policies, and system misconfigurations

Each vulnerability was assigned a **CVSS score**, mapped to a **CWE category**, and included appropriate remediation guidance

2.6 IMPACT ANALYSIS

If left unpatched or unresolved, the identified vulnerabilities could lead to

- Unauthorized access

2. VA USING VARIOUS VA TOOLS

- Data leakage
- Privilege escalation
- Denial of service
- Compromise of entire system or application
- Loss of confidentiality, integrity, and availability

2.7 RECOMMENDATIONS

- Apply all missing security patches and updates.
- Disable unnecessary services and close unused ports.
- Use strong password and account policies.
- Update outdated web components and sanitize user inputs.
- Implement proper access control and session management.
- Regularly perform vulnerability assessments and patch management processes

2.8 CONCLUSION

The vulnerability assessment using Smart Scanner, Nessus, Acunetix, and MBSA provided a comprehensive understanding of the security posture of the target environment. The combination of multiple tools ensured high accuracy, minimized false positives, and covered network, system, and application-level vulnerabilities. Implementing the recommended remediation steps will significantly reduce the attack surface and enhance the overall security resilience

3. VA USING AI

3. VA Using AI

This report provides an overview of performing a **Vulnerability Assessment using AI-powered Gemini commands integrated with the Kali Linux terminal**. The objective was to leverage AI assistance to automate information gathering, analyze vulnerabilities more efficiently, and enhance decision-making during security assessments. Using AI Gemini allows security professionals to generate commands, analyze scan results, and interpret CVEs more intelligently within the Kali environment.

3.1 OBJECTIVE

- To perform a structured vulnerability assessment using **AI Gemini integrated in the Kali terminal**.
- To automate verbosity, interpretation, and analysis of vulnerability scan outputs.
- To enhance traditional VAPT (Vulnerability Assessment & Penetration Testing) workflow using AI-guided commands.
- To identify vulnerabilities, misconfigurations, outdated services, and exposed ports using AI-assisted Nmap, Nikto, and other tools

3.2 SCOPE

- Kali Linux (latest version) with AI Gemini CLI installed.
- Target: Internal test machine / virtual machine.
- Tools used (AI-assisted through Gemini):
 - **Nmap** (Port scanning and service detection)
 - **Nikto** (Web server scanning)
 - **OpenVAS** (High-level vulnerability identification)
 - **Searchsploit** (Exploit mapping)
 - **Gemini CLI** for AI-assisted interpretation

3.3 METHODOLOGY

AI-Assisted Reconnaissance

AI Gemini was used to generate optimized scanning commands

"Give best Nmap command for full port + service + OS detection"

3. VA USING AI

```
(sai@kali)-[~]
$ gemini-cli

> GEMINI

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. Create GEMINI.md files to customize your interactions with Gemini.
4. /help for more information.

> Give best Nmap command for full port + service + OS detection

+ A powerful and commonly used Nmap command for a comprehensive scan is:

1 sudo nmap -p- -sV -sC -O -T4 -oA <scan_name> <target>

Here's a breakdown of what each part does:

* sudo: Required for OS detection (-O) and other privileged scans.
* -p-: Scans all 65535 TCP ports.
* -sV: Probes open ports to determine service and version information.
* -sC: Runs a set of default, safe scripts for vulnerability detection and further enumeration.
* -O: Enables operating system detection.
* -T4: Sets the timing template to "Aggressive" for a faster scan.
* -oA <scan_name>: Saves the output in all major formats (Normal, XML, and Grepable) with the specified file name (e.g., scan_name.nmap, scan_name.xml, scan_name.gnmap).
* <target>: The IP address or hostname you want to scan.

Note: This scan is very thorough but also very "loud" and easily detectable by firewalls and intrusion detection systems. Always ensure you have permission to scan the target.

>  Type your message or @path/to/file

/usr/lib/gemini-cli          no sandbox (see /docs)          gemini-2.5-pro (96% context left)
```

fig. 3.3.1 Using Gemini-cli for searching for the command.

Port & Service Discovery

Nmap scan performed with Gemini-optimized commands

- Identified open TCP/UDP ports.
- Detected service versions.
- Collected OS fingerprinting data

cat nmap.txt | gemini "Explain critical vulnerabilities and misconfigurations"

3. VA USING AI

```
> cat nmap.txt | gemini "Explain critical vulnerabilities and misconfigurations"

• Based on the Nmap scan results, here is an analysis of critical vulnerabilities and misconfigurations.

Summary

The scan indicates a CRITICAL security posture. Several services are outdated and exposed, making the system highly vulnerable to attack. The most severe issues include a known backdoor in the FTP service and the use of unencrypted protocols for management.

—

Critical Vulnerabilities & Misconfigurations

These are the most severe issues and should be addressed immediately.

1. Port 21/tcp (FTP) - vsftpd 2.3.4
* Vulnerability (CRITICAL): Version 2.3.4 of vsftpd contains a well-known, malicious backdoor (CVE-2011-2523). An attacker can gain a command shell on the server by entering a specific string in the username field, leading to a full system compromise.
* Misconfiguration (High): The service allows Anonymous FTP login. This permits anyone on the network to connect to the FTP server without a password, potentially allowing them to upload malicious files or download sensitive data.
* Remediation:
  * Immediately take the server offline and upgrade or replace the vsftpd service. Do not simply patch it; assume the system may already be compromised and conduct a full investigation.
  * Disable anonymous FTP access in your FTP server's configuration.

2. Port 23/tcp (Telnet) - Linux telnetd
* Misconfiguration (CRITICAL): The Telnet protocol is unencrypted. All communication, including usernames and passwords, is sent in cleartext. An attacker sniffing network traffic can easily capture these credentials and gain access to the server.
* Remediation:
  * Immediately disable the Telnet service.
  * Use SSH (Secure Shell) on port 22 for all remote management, as it provides strong encryption.

3. Port 22/tcp (SSH) - OpenSSH 5.9p1
* Vulnerability (High): OpenSSH version 5.9 is over a decade old and has numerous documented vulnerabilities, including weaknesses that could allow an attacker to bypass security restrictions or enumerate valid usernames (e.g., CVE-2018-15473).
* Remediation:
  * Upgrade the OpenSSH server to the latest stable version provided by your distribution's package manager.
  * Ensure your SSH configuration disallows root login and uses modern, strong cryptographic algorithms.

4. Port 80/tcp (HTTP) - Apache httpd 2.2.22
* Vulnerability (High): This version of Apache is end-of-life and vulnerable to many exploits, including denial-of-service (like Slowloris), cross-site scripting (XSS), and information disclosure bugs.
* Misconfiguration (Low): The server is running with a default "It works!" page. This confirms the server is not properly configured and leaks its software version, which helps an attacker tailor their attack.
* Remediation:
  * Upgrade the Apache server to a modern, supported version (e.g., 2.4.x).
  * Remove the default page and deploy your actual web application.
  * Configure the server to hide its version number and other sensitive information in HTTP headers.
```

fig. 3.3.2 Getting Information about the misconfigurations

3.4 FINDINGS / RESULTS

- Multiple open ports (e.g., 22, 80, 139, 445, 3306).
- Outdated service versions with known CVEs.
- Weak security configurations (e.g., directory listing enabled, SMB outdated version).
- Missing patches and default credentials (if applicable).
- Potential web vulnerabilities such as:
 - XSS indicators
 - Directory traversal
 - Information disclosure

3.5 IMPACT

If exploited, the identified vulnerabilities may lead to:

3. VA USING AI

- Unauthorized access to the system
Sensitive information exposure
- Privilege escalation
- Remote Code Execution
- Lateral movement across the internal network

3.6 RECOMMENDATIONS

AI-generated recommendations included

- Patch all outdated services immediately.
- Disable unnecessary open ports.
- Apply strong authentication on exposed services.
- Update Apache/SMB/FTP services to secure versions.
- Implement firewall rules to restrict access.
- Regularly use vulnerability scanners combined with AI analysis for faster remediation

3.7 CONCLUSION

Using **AI Gemini in the Kali Linux terminal significantly enhanced the vulnerability assessment process** by:

- Optimizing scan commands
- Automating interpretation of results
- Mapping vulnerabilities to CVEs
- Providing instant remediation guidance
- Reducing manual effort in report writing

This demonstrates how integrating AI with traditional VAPT workflows improves efficiency, accuracy, and reporting quality